# UNIVERSIDAD DIEGO PORTALES

FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA EN INFORMÁTICA Y TELECOMUNICACIONES




## Tarea 2 Sistemas Operativos

**Integrantes**
Carlos Beaudout

# 1. Descripción general

El programa implementa un torneo de Komikan en el que cada partida enfrenta a un Puma contra un equipo de doce perros. El servidor Matchmaker recibe una cantidad $N$ de clientes y forma parejas para iniciar partidas. Si $N$ es impar, un cliente queda sin pareja. El parámetro $M$ establece la cantidad máxima de partidas que pueden ejecutarse simultáneamente.

Cada partida utiliza un tablero independiente y crea trece hilos para representar al Puma y a los doce perros. Estos hilos realizan movimientos concurrentes, respetando las reglas y las posiciones disponibles. Además, cada partida cuenta con un juez que supervisa su estado y determina cuándo debe finalizar. Este juez se ejecuta dentro del trabajador que administra la partida.

El tablero contiene 28 posiciones: las 25 casillas de la cuadrícula, el trono y las dos intersecciones superiores. Las posiciones iniciales y los retardos de movimiento se pueden configurar mediante un archivo externo.

# 2. Mecanismos de sincronización

Se utilizan los siguientes mecanismos para coordinar la ejecución concurrente:

- **Mutex (`pthread_mutex`):** protege el tablero compartido y las variables relacionadas con las posiciones, capturas y estado de la partida. Así, dos hilos no pueden modificar simultáneamente el mismo tablero. También se utiliza otro mutex para evitar que las salidas de distintas partidas se mezclen en la consola.
- **Variable de condición (`pthread_cond`):** permite que el juez espere cambios en el estado de la partida y sea notificado cuando ocurre un movimiento o se determina un resultado. Se utiliza una espera temporizada de 100 milisegundos para comprobar periódicamente si se recibió una interrupción.
- **Semáforo (`sem_t`):** se inicializa con $M$ permisos. Cada trabajador obtiene un permiso antes de ejecutar una partida y lo devuelve cuando esta termina. Además, el programa crea como máximo $M$ trabajadores del Matchmaker, manteniendo así el límite de partidas simultáneas.
- **Hilos (`std::thread`):** permiten ejecutar concurrentemente los movimientos de las trece piezas y el trabajo de los distintos trabajadores del Matchmaker.

Los movimientos utilizan retardos aleatorios dentro del rango establecido en la configuración. Después de esperar, cada hilo adquiere el mutex, comprueba los movimientos válidos y actualiza el tablero antes de liberar el recurso.

# 3. Deadlock, livelock y starvation

El mutex evita que varios hilos modifiquen el tablero al mismo tiempo. Las secciones críticas son acotadas y los retardos de los movimientos se realizan fuera del mutex. Además, se utiliza un mecanismo de liberación automática del bloqueo al salir de la sección crítica. Estas medidas reducen la posibilidad de interbloqueo (*deadlock*).

El *livelock* ocurre cuando los hilos continúan reaccionando entre sí sin conseguir avanzar. En esta implementación, cada pieza espera un retardo antes de intentar realizar un movimiento y valida su acción bajo protección del mutex. Sin embargo, esto no constituye una garantía formal de que el livelock sea imposible.

La *starvation* ocurre cuando un hilo espera indefinidamente para acceder a un recurso. Aunque el mutex garantiza exclusión mutua, no asegura por sí solo un orden justo de acceso. Por lo tanto, no se puede garantizar que todos los hilos tengan exactamente la misma oportunidad de ejecución. Las secciones críticas breves ayudan a reducir los tiempos de espera.

# 4. Manejo de fallos e interrupciones

El programa contempla fallos inesperados durante la ejecución de una partida. Si una pieza produce una excepción, el hilo correspondiente captura el error, actualiza el estado de la partida como abandono y permite que el oponente sea declarado ganador. Si el fallo corresponde al Puma, ganan los perros; si corresponde a un perro, gana el Puma.

También se utiliza un mutex robusto, configurado mediante `PTHREAD_MUTEX_ROBUST`. Si un hilo termina inesperadamente mientras mantiene el mutex, otro hilo puede recibir el resultado `EOWNERDEAD` al intentar adquirirlo.

En ese caso, se identifica la pieza que estaba dentro de la sección crítica, se registra el abandono y se utiliza `pthread_mutex_consistent()` para recuperar el mutex y continuar con la finalización de la partida.

Este mecanismo permite recuperarse de la terminación de un hilo que deja el mutex bloqueado. No permite recuperar la ejecución si termina el proceso completo, por ejemplo, ante un fallo que cierre todo el programa.

Al recibir `Ctrl+C`, el servidor deja de iniciar nuevas partidas y las partidas activas reciben la señal de detención mediante una variable compartida. El juez comprueba periódicamente esta condición y finaliza la partida. Luego, los hilos se esperan mediante `join` y se liberan los mutexes, las variables de condición y el semáforo utilizados.

## 5. Rendimiento y concurrencia

El parámetro $M$ controla el nivel máximo de paralelismo entre partidas. Si $M = 1$, solo puede ejecutarse una partida a la vez. Si $M > 1$, distintas partidas pueden avanzar simultáneamente hasta alcanzar ese límite. El programa crea como máximo $M$ trabajadores del Matchmaker, y cada uno administra una partida a la vez.

Aumentar $M$ puede elevar el rendimiento total (*throughput*), ya que permite completar más partidas en un mismo intervalo. Sin embargo, también aumenta el consumo de recursos y la competencia por la CPU.

La latencia de una partida corresponde al tiempo transcurrido desde que se inicia hasta que se declara un resultado. Depende de los retardos aleatorios, de la cantidad de movimientos necesarios y de la espera para acceder a los recursos compartidos. Dentro de cada partida, los hilos de las piezas pueden competir por el mutex, pero las secciones críticas son breves para reducir esa espera.

No se presentan valores numéricos de throughput o latencia, ya que estos requieren mediciones experimentales bajo una configuración y carga determinadas. Cualitativamente, la implementación permite controlar la cantidad de partidas simultáneas mediante el parámetro $M$.

## 6. Conclusión

La solución utiliza hilos para representar la actividad concurrente de las piezas y de las partidas, junto con mutexes, variables de condición y semáforos para coordinar el acceso a los recursos compartidos. El juez supervisa cada partida y permite finalizarla ante una victoria, un abandono o una interrupción del servidor.

El uso de mutex robusto agrega un mecanismo de recuperación ante la terminación inesperada de un hilo que deja bloqueado el tablero. Finalmente, el parámetro $M$ permite controlar el paralelismo del torneo y estudiar el equilibrio entre rendimiento y uso de recursos.